



Project Report

CSE-326: Microprocessor and Microcontroller Lab

Topic: Car Parking System using Arduino UNO

Submitted By:

Name	ID	Intake	Section
Md. Monirul Islam	20234103338	52	09
MD.Jahidul Islam Shihab	20234103347	52	09
MD. Al Nasir Uddin Siam	20234103349	52	09
Y easir Ibna Hasibur Rahman	20234103358	52	09

Supervised By :

Shefayatuj johara Chowdhury

Lecturer

Department of CSE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY (BUBT)

April 21,2026

Abstract

The Smart Car Parking System is an intelligent and automated solution developed to optimize parking space utilization and reduce traffic congestion in urban areas. The system integrates sensors to detect vehicle presence and continuously monitor slot availability in real time. A microcontroller-based unit processes the collected data and controls access through an automated barrier mechanism using a servo motor.

User interaction is enhanced through an LCD display that provides real-time parking availability, while LED indicators assist drivers in quickly locating vacant spaces. By minimizing manual intervention, the system reduces waiting time, fuel consumption, and human error. Additionally, the design is scalable and can be extended with advanced features such as IoT-based remote monitoring, mobile application integration, and secure access control.

Overall, this system provides a reliable, cost-effective, and efficient approach to modern parking management, contributing to smarter and more sustainable urban infrastructure.

Abbreviations

Abbreviation	Full Form
MCU	Microcontroller Unit
IDE	Integrated Development Environment (referring to Arduino IDE)
IR	Infrared (Sensor)
LCD	Liquid Crystal Display
I2C / PC	Inter-Integrated Circuit (Communication Protocol)
PWM	Pulse Width Modulation
LED	Light Emitting Diode
RFID	Radio Frequency Identification
IoT	Internet of Things
USB	Universal Serial Bus
DIP	Dual In-line Package
I/O	Input / Output
GND	Ground
VCC	Voltage at the Common Collector (Positive Power Supply)
VSS	Voltage Source Supply (Ground)
VDD	Voltage Drain Drain (Positive Power Supply)
MILP	Mixed Integer Linear Program
POS	Point of Sale
Trig	Trigger (Ultrasonic Sensor Pin)
Echo	Echo (Ultrasonic Sensor Pin)
BDT	Bangladeshi Taka

TABLE OF CONTENTS

Abstract	i
Abbreviations	ii
TABLE OF CONTENTS	iii-iv
List of Figures	v
Chapter : 1. Introduction	1-3
1.1 Background of study and motivation	1
1.2 Project objective	1
1.3 Report outlines	2
1.4 Our Contributions	2-3
Chapter : 2. Literature review	4-5
2.1 Introduction	4
2.2 Historical Development of Parking Automation	4
2.3 Advancement in Sensor-Based Detection	4
2.4 Conclusion	4-5
Chapter : 3. Methodology and modeling	6-19
3.1 Introduction	6
3.2 Block diagram of the proposed project	6-7
3.3 Construction and working principle	7-8
3.3.1 Process of work	9
3.4 Hardware Design	9
3.5 Description of the component	10
3.5.1 Arduino Board	10

3.5.2	LED	11
3.5.3	16*2 LCD display	11-12
3.5.4	Servo motor	12
3.5.5	IR sensor	12-13
3.5.6	Others component	13
3.6	Flow Chart of the system	14
3.7	Implementation	14-16
3.8	Test/Experimental setup	16-17
3.9	Cost Analysis	18
3.10	Problem statement	19
Chapter : 4.	Result and discussion	20-22
4.1	Hardware connecting	20
4.2	Simulation/Numerical Analysis	21
4.3	Measured response/Experimental result	21
4.4	Advantage of smart car parking system	22
4.5	Disadvantage of smart car parking system	22
4.6	Benefit analysis of smart car parking system	22
Chapter : 5.	Conclusion and future work	23
5.1	Conclusion	23
5.2	Future work	23
	References	24
	Appendix	25-30

LIST OF FIGURES

Figure No.	Figure Name	Page
1	Block Diagram of car parking system	7
2	Entry part of car parking system	8
3	Exit part of car parking system	8
4	Smart Car Parking System working Process	9
5	Arduino UNO	10
6	3mm round LED	11
7	16x2 Blue Display	12
8	Servo Motor	12
9	IR sensor	13
10	Breadboard,wires and 12 C module	13
11	Flow chart of Car Parking System	14
12	The design of Car Parking System using Arduino UNO	15
13	Uploaded code in Arduino IDE	
14	Connecttion between Arduino and others	19
15	Actual Working Model	20
16	Parking Process	24
17	Parking Area	24
18	Full Parking Area	25

Chapter-1: Introduction

1.1 Background of study and motivation:

Parking is a major problem in our country. Drivers in our country cannot properly park their vehicle in the parking slot because of improper parking system. Sometimes when they do not find any parking space, they park their vehicle in the roadside and as a result roads become so narrow and occurs traffic jam. Also, sometimes it seems that people start fighting for parking slot in the parking lot. Because from outside people cannot understand whether there is any parking slot is available or not. It also wastes our lot of time. Till now no smart technology exists to solve this problem. This project will change and standardize the parking system for a parking lot.

An automated car parking system is a process through which car parking can be done more efficiently and easily than manual method. The system will provide the user better services. The system counts the number of cars in the garage and checks if there's any vacancy. There's an entry and exit path. When vehicle enters, the display shows the number of cars inside. When any vehicle leaves, the count decreases and shown on display. If the garage is full. The display will show a message regarding that. This whole process includes the use of Arduino, Display and sensor. The sensor detects whether the vehicle is entering or leaving. The report then showed on display.

1.2 Project Objectives:

The objectives to be achieved are:

- Providing safe and secure parking spots within a limited space.
- Developing a sophisticated automated parking system that lowers workforce, and traffic congestion and save time.
- Automatically keeping count and monitoring total entered and exited vehicle and calculate free available slots.
- Implementing a digital payment method for parking system.

1.3 Project Scope

The scope of this project is centered on creating a functional, small-scale prototype of an automated parking management system. The boundaries and reach of this project include:

- **Real-time Monitoring:** The system is designed to monitor individual parking slots using **IR sensors** to provide immediate feedback on occupancy.
- **Automated Entry/Exit Control:** Using **Ultrasonic sensors** at the entrance and exit points to detect vehicles and trigger a **Servo motor-based gate** mechanism.
- **User Information System:** Integrating a **16x2 LCD display** to provide drivers with live data regarding the "Total Spaces" and "Available Spaces," preventing entry when the lot is full.
- **Hardware Efficiency:** Utilizing the **Arduino UNO** as the central processing unit to demonstrate a cost-effective alternative to expensive, high-end parking solutions.
- **Safety and Automation:** Reducing human intervention in traffic management within the parking area, thereby decreasing the likelihood of minor accidents and congestion.

1.4 Our Contributions

Our work in this project involves the design, integration, and testing of a multi-sensor parking solution. Our specific contributions include:

- **Hybrid Sensor Integration:** We successfully implemented a dual-sensor approach, using **Ultrasonic sensors** for reliable gate triggering and **IR sensors** for precise slot-level monitoring, ensuring a higher level of accuracy than single-sensor systems.
- **Automated Logic Development:** We developed the Arduino firmware logic that calculates real-time availability by processing data from multiple sensor inputs and controlling the servo motor movement accordingly.
- **Circuit Optimization:** We designed a compact circuit layout using a breadboard and jumper wires that efficiently powers and connects the LCD, sensors, and actuators to a single Arduino UNO.
- **User Interface Design:** We programmed the LCD display to provide clear, user-friendly messages such as "Space Left" and "Parking Full," improving the overall user experience for the driver.

- **Comparative Research:** We conducted a detailed literature review comparing modern technologies like Image Processing and RFID with our sensor-based approach to justify the cost-effectiveness and reliability of our prototype.

Chapter-2: Literature review

2.1 Introduction

Modern urban management faces a critical challenge in parking logistics. Drivers often struggle to locate vacant spots, leading to traffic congestion and fuel waste. The following review examines how various researchers have addressed this problem using automated systems, from early image-processing techniques to modern IoT-integrated sensor networks.

2.2 Historical Development of Parking Automation

1. Number Plate Recognition (2012): Early research by Rashid et al. focused on electronic fee collection through license plate detection using complex image-processing algorithms.
2. RFID-Based Access (2020): Researchers in Indonesia simulated portal door systems using RFID cards as subscriber IDs to verify identities and automate entry and exit.
3. Mobile and Cloud Integration: Studies by Ba Sabeen and Bonde et al. (2018) highlighted the shift toward mobile application-based control, allowing for remote slot reservation and automated parking management without human staff.

2.3 Advancement in Sensor-Based Detection

- Ultrasonic Sensor Utility: Research by Wu et al. (2016) and Mohapatra (2020) established the reliability of ultrasonic waves for obstacle detection and slot fill-level measurement. In our project, these sensors are utilized to track vehicle entry and exit at the gate.
- IR Sensor Integration: IR sensors are frequently cited for their effectiveness in short-range presence detection. Our system utilizes these at each parking slot to provide real-time status updates to the Arduino.
- Internet of Things (IoT) Synergy: Basavaraj (2015) and Khanna (2016) demonstrated that connecting environmental things to a network via IoT can drastically reduce the time spent finding a parking lot, thereby lowering carbon footprints.

2.4 Conclusion

The literature indicates that while high-end technologies like Image Processing and Mixed Integer Linear Program (MILP) models offer advanced allocation, they are often computationally heavy. There is a clear demand for low-cost, reliable, and sensor-based systems that provide real-time information locally. Our project addresses this by combining the gate automation of Ultrasonic sensors with the individual slot-tracking capabilities of IR sensors , coordinated by an Arduino UNO

Chapter-3: Methodology and Modeling

3.1 Introduction

This chapter provides a detailed explanation of the methodology used to design and implement the Smart Car Parking System. The methodology encompasses the overall approach, including hardware integration, logic development, and the strategies used to collect and process real-time data. The primary goal of this "Smart Parking" methodology is to create an edge technology that assists drivers in locating empty spots effectively and autonomously. By integrating specialized sensors with an Arduino UNO microcontroller, the system provides a robust and cost-effective solution for urban parking management.

Key aspects of this methodology include:

1. Presence Detection: Utilizing IR sensors at each individual parking space to identify the occupancy status of the slot.
2. Flow Monitoring: Implementing Ultrasonic sensors at the entry and exit points to track the number of vehicles and trigger the automated gate mechanism.
3. Centralized Control: Using the Arduino UNO as the "brain" to process sensor inputs and coordinate the output devices like the Servo motor (gate) and 16x2 LCD display
4. Real-time Feedback: Ensuring the system updates available slots instantly to provide drivers with accurate information, thereby reducing searching time and fuel consumption.

3.2 Block Diagram of the proposed project

The circuit shown in Fig. 1 an Arduino microcontroller (MCU) requires only 5V. As Arduino Uno has an inbuilt 5V voltage regulator, a common 5V supply can be used for the whole system.

The brain of the circuit is Arduino Uno MCU board (BOARD1). It is based on Arduino UNO R3 V1.0 and has 14 digital input/output (I/O) pins, six analogue inputs, servo motor, a USB connection, IR sensor with 12 C module among others. It can be programmed using Arduino IDE software IR sensor are connected to Arduino digital pins 2 and 3 and servo motor are connected to Arduino pins 13, which are used for serial communication.

The LCD display (LCD1) is used to display messages during action. Here, a 16×2 display is used; each character is made of 5×7 dot-matrix. Pins 3, 4, 5 and 6 of the LCD are the control lines connected to preset (PR1) output, pin 12 (Arduino), GND and pin 11 (Arduino). Pins 11,12, 13 and 14 are data pins of the LCD that are connected to pins 7, 6, 5 and 4 of Arduino, respectively. Preset PR1 is used to adjust the contrast of the LCD display.

Block Diagram of our proposed system is given below:

Block Diagram

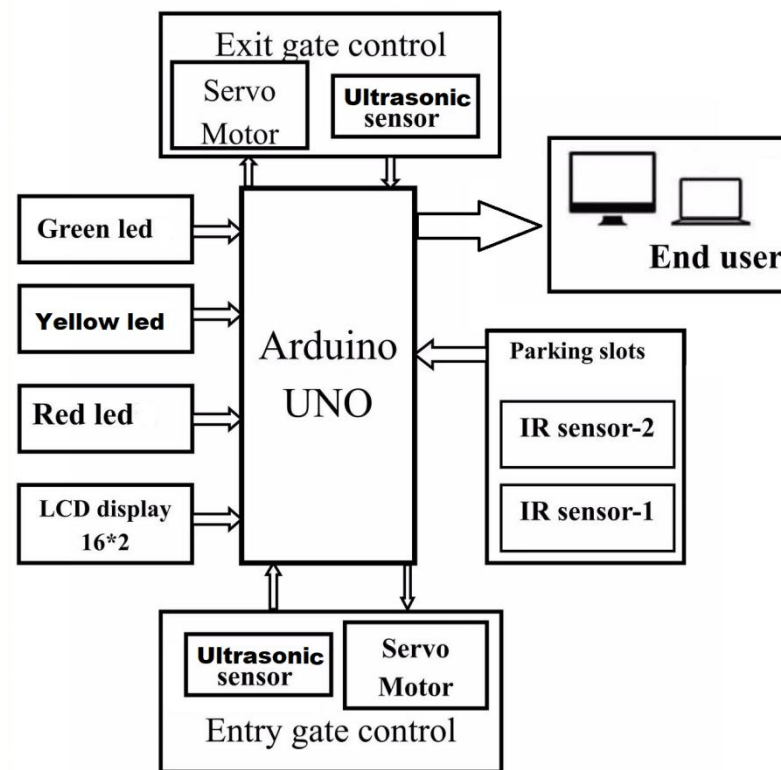


Figure-1: Block Diagram of car parking system

3.3 Construction and working principle:

The construction and working of the project are divided into two parts:

1) Entry Part:

The project is a sensor based automatic system. The Entry Part of the project consists of Arduino Uno microcontroller to which a servo motor, IR sensor, LCD and ultrasonic sensor are interfaced. The servo motor acts as a gate at the entrance and it opens and closes when the ultrasonic sensor detects presence of car. The input IR sensors reads presence of any car and sends to the controller Arduino UNO board. While entry ultrasonic sensor detect car, it opens the door in condition of having available slot. After entering the car, the gate gets closed with

the help of servo motor. The motor gets instruction from the board. The LCD displays the parking slots which are empty for the car drivers. While A car gets into a slot, the corresponding slot status in the LCD display shows "Have space". At the time of exiting a slot, the status of the slot changed to "Vacant". When all the slots are booked, the system doesn't allow the fourth car to enter, and shows "Sorry parking full". The IR sensors detect the presence and absence of car in each parking slot.

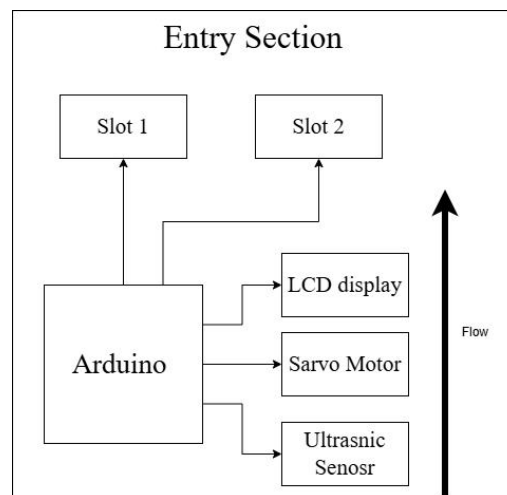


Figure-2: Entry part of car parking system

2) Exit Part

The Exit Part of the project consists of Arduino Uno to which a servo motor, Ultrasonic sensor and the object counter are interfaced. The servo motor acts as a gate at the entrance and it opens and closes when the ultrasonic sensor detects presence of car. When a car exits from a slot, the exit ultrasonic sensor detects the car and opens the gate with the servo motor. Then total available free slot count increase by 1 in the screen.

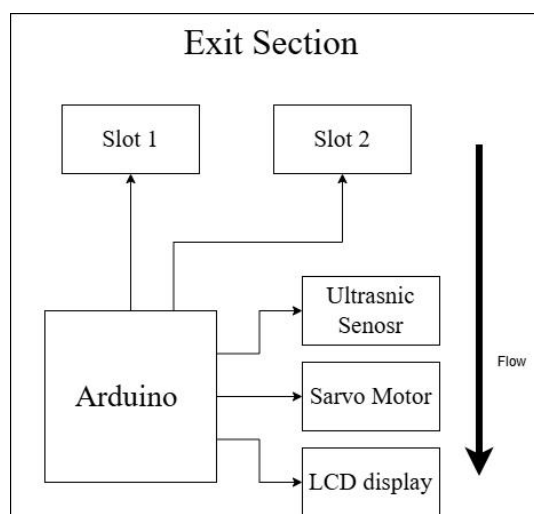


Figure-3: Exit part of car parking system

The system will be very reliable and understandable to the users.

3.3.1 Process of Work:

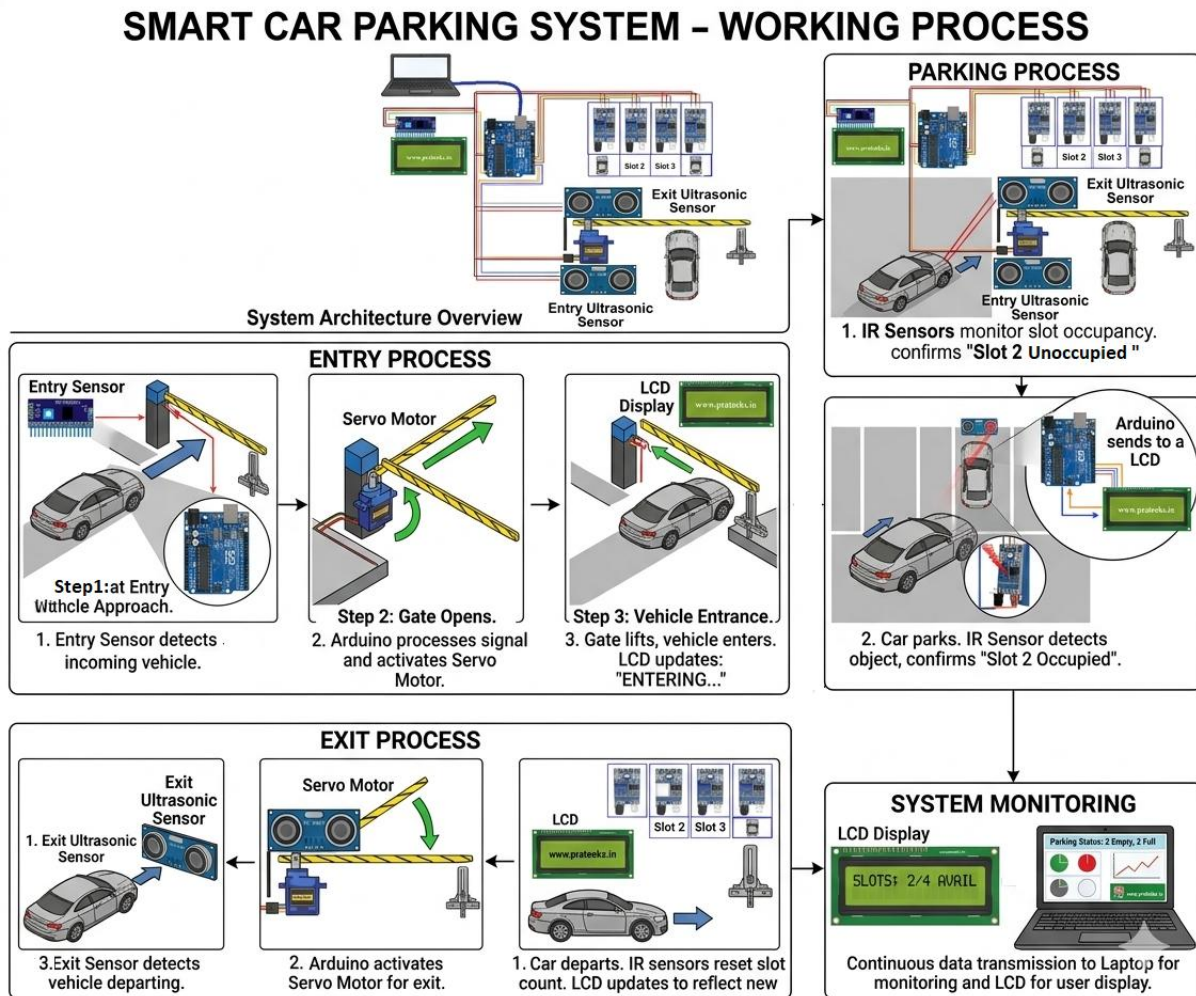


Figure-4: Smart Car Parking System working Process

3.4 Hardware Design

Hardware equipment that we need in order to build the project are given below:

1. Arduino UNO
2. Ultrasonic Sensor
3. LCD Screen
4. Bread Board
5. Sarvo Motor
6. Connecting Wires
7. 12 C module

3.5 Description of the important component:

First of all, we use Proteus software to complete the simulation. in this simulation, we've used some important components to build the whole parking system, which are Arduino UNO R3 V1.0, LEDGREEN, LM044L 20x4 Alphanumeric Display, MOTOR-PWMSERVO, POT variable resistor, IR sensor with module.

3.5.1 Arduino Board

Arduino is an open-source electronics platform based on flexible, easy-to-use hardware and software. Unlike competitors like Raspberry Pi or Parallax, Arduino is highly accessible due to its massive community and "Creative Commons" licensing.

- **Arduino UNO:** The most popular board for beginners. It features the *ATmega328* microcontroller, 14 digital I/O ports (6 with PWM), and 6 analog inputs. A key benefit is the removable DIP-package chip, which allows for easy replacement if the controller is damaged.
- **Arduino Leonardo:** Uses the *ATmega32u4*, which has built-in USB communication. This allows the board to be recognized by a PC as a native keyboard or mouse.

Key Advantages:

- **Low Cost:** Highly affordable compared to other microcontrollers.
- **Cross-Platform:** Software runs on Windows, macOS, and Linux.
- **Simple Programming:** Uses a simplified version of C++ that is easy for beginners yet powerful for professionals.



Figure-5: Arduino UNO

3.5.2 LED

LED lights are called light-emitting diodes. They are highly efficient solid-state semiconductor devices that convert electricity directly into light, offering long-lasting, durable, and energy-efficient lighting solutions for homes, electronics, and commercial applications.

Common packaging technologies include SMD (Surface Mount Device) and RGB (Red-Green-Blue) for color-changing lights



Figure-6: 3mm round LED

3.5.3 16x2 Alphanumeric Display

An LCD (Liquid Crystal Display) is an inexpensive module used to display text and custom characters.

- **Capacity:** 16 characters per line across 2 lines (32 characters total).
- **Resolution:** Each character is formed by a 5×8 pixel matrix.
- **Interface:** Supports both 8-bit and 4-bit (saving I/O pins) data transfer modes.

Pin Configuration Summary:

- **VSS/VDD:** Ground and Power (5V).
- **V0:** Contrast adjustment (usually via a potentiometer).
- **RS (Register Select):** Toggles between Command and Data modes.
- **RW (Read/Write):** Typically grounded for "Write" mode.
- **E (Enable):** Triggers the execution of data/commands.
- **D0–D7:** Data bus pins.
- **A/K:** Backlight power (Anode/Cathode).



Figure-7: 16x2 Blue Display

3.5.4 Servo Motor

A Servo Motor is a closed-loop device that uses feedback to control the precise angular position of an output shaft.

- **Mechanism:** Consists of a motor, gear train, and control circuit. It typically rotates **180°** based on Pulse Width Modulation (PWM) signals.
- **Working Principle:** The internal controller compares a reference input signal to the current position (feedback) and adjusts the motor until the error is zero.
- **Software:** Arduino provides a built-in **Servo Library** to simplify control.



Figure-8: Sarvo Motor

3.5.5 IR Sensor

Infrared sensors detect and measure IR radiation (heat) in the environment.

- **Module Components:** Includes an IR Emitter (Tx), Receiver (Rx), Op-Amp, and a trimmer pot for sensitivity adjustment.
- **Specifications:** Operates at 5V with a detection range of up to 20cm.

Types of IR Sensors:

- **Active:** Contains both a transmitter and receiver (e.g., obstacle sensors). It emits energy and detects the reflection.
- **Passive (PIR):** Only contains a receiver. It detects IR radiation emitted by external objects (e.g., motion detectors).

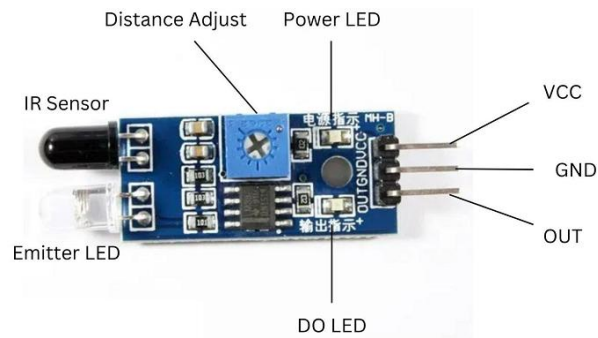


Figure-9: IR sensor

3.5.7 Other Components

- **Breadboard:** Used for solderless prototyping and temporary circuit connections.
- **Connecting Wires:** Jumper wires (Male-to-Male, Male-to-Female) used to route signals between the Arduino and peripheral modules.
- **12 C module :** It helps display to connect properly with arduino.

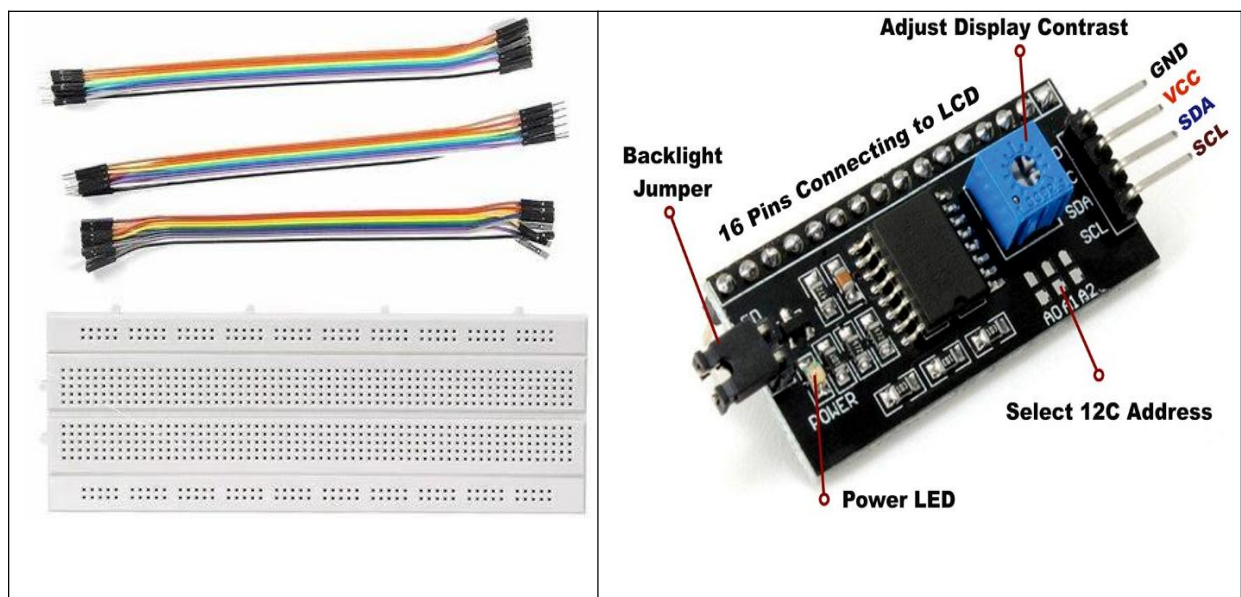


Figure-10: Breadboard,wires and 12 C module

3.6 Flow Chart

The flow chart includes how the system works. The program flow chart is given below:

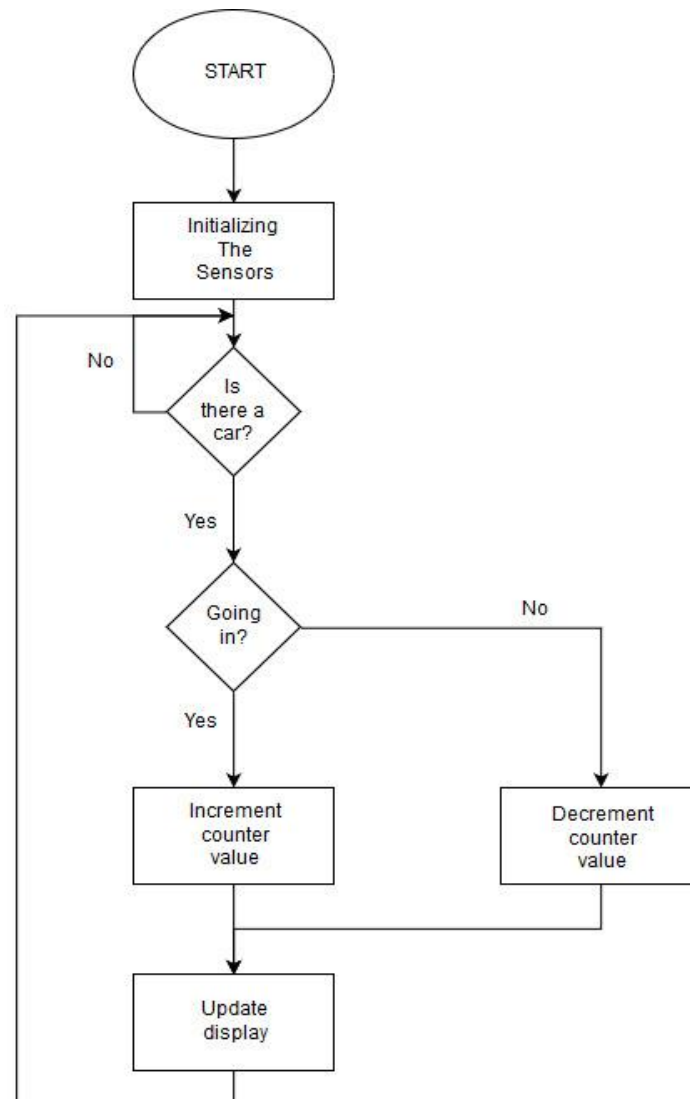


Figure-11: Flow chart of Car Parking System

3.7 Implementation:

To implement the Smart Parking System, we have used Proteus professional 8.12 simulating software and Arduino UNO. Figure 9 shows the simulation implementation of the project.

Firstly, A new project created without any firmware in proteus. After creating the blank new project, there will be a schematic diagram window. In this schematic window, the simulation circuit is constructed. All of the devices used here, were found under pick device option. Arduino UNO R3 V1.0 was chosen as microcontroller board for Car Parking System. This system works on the basis of some major components such as Arduino UNO R3, LED-

GREEN-YELLOW-RED,IR-Sensor,Ultrasonic Senosrs,Sarvo Motor,16x2 Alphanumeric Display had used in this project. The display was connected to the board through the expander. So basically, we have used the 16x2 Alphanumeric Display to print the slot list, each slot status of the parking lot. For the connection of the Display, we used Remote 8-bit I/O expander to reduce the connection with the Arduino pin. As it is a simulation-based project, here IR sensor use was not too easy. The logic states used to show the functionalities of the IR sensors. Every logic states connected to individual digital I/O pin of the Arduino UNO.

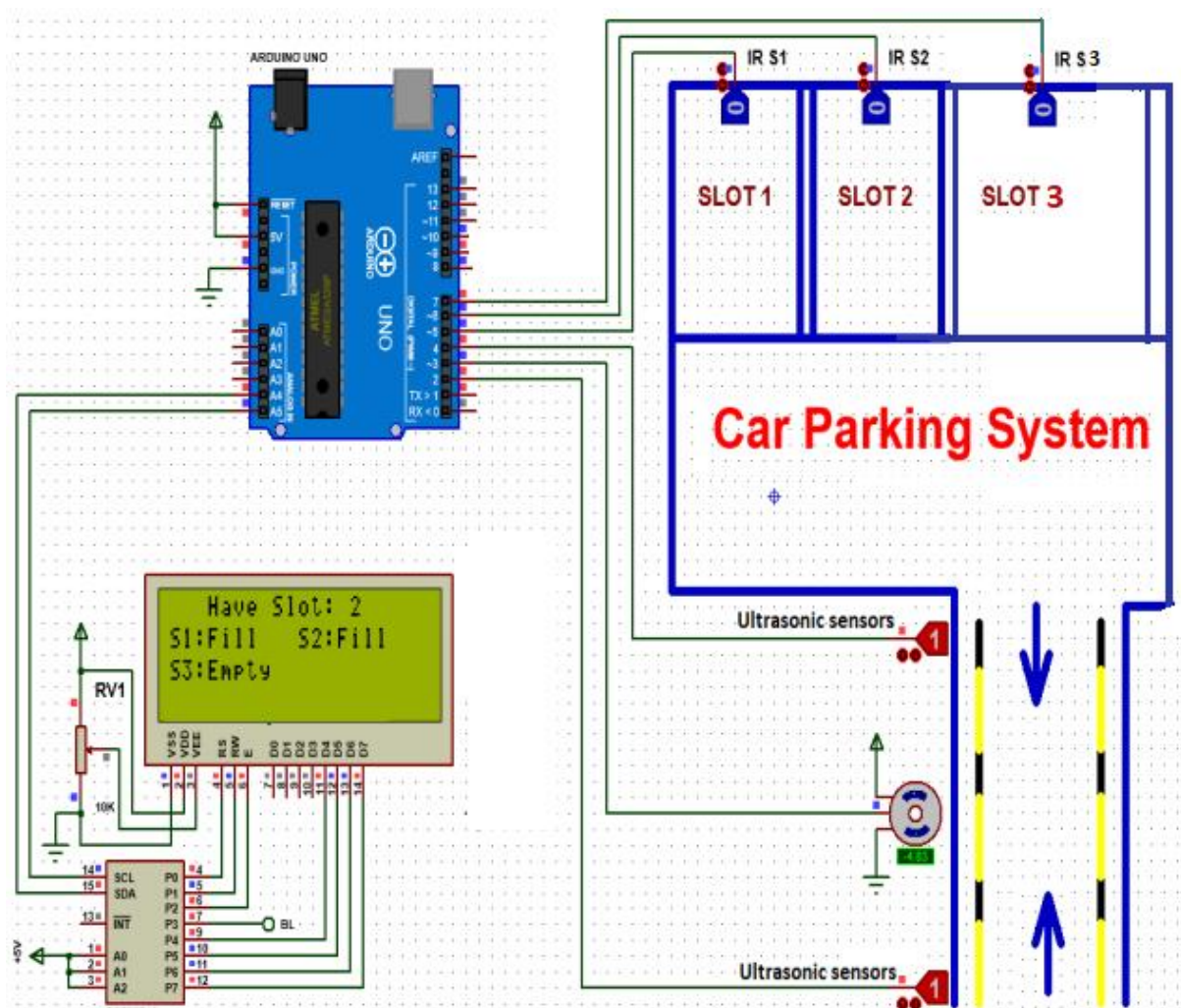


Figure 12: The design of Car Parking System using Arduino UNO

The servo motor was also connected with a digital pin for get instruction and it get external power to run. Two IR sensor set at the gate of the parking spot for detect entry and exit of car. Each slot also has IR sensor to identify slot is empty or not. The servo motor rotates 180 degrees to open the gate and 90 degrees to close the gate. The green LED lights while the whole system is turned on. Pot variable resistor ensures different power supply to the LCD

display. After setting up the components as figure 9, we went to Arduino IDE and installed all the necessary libraries to run the project successfully. Then the codes/commands were written and uploaded to the Arduino Board. After upload finely, the code worked without any errors.

3.8 Test/Experimental setup:

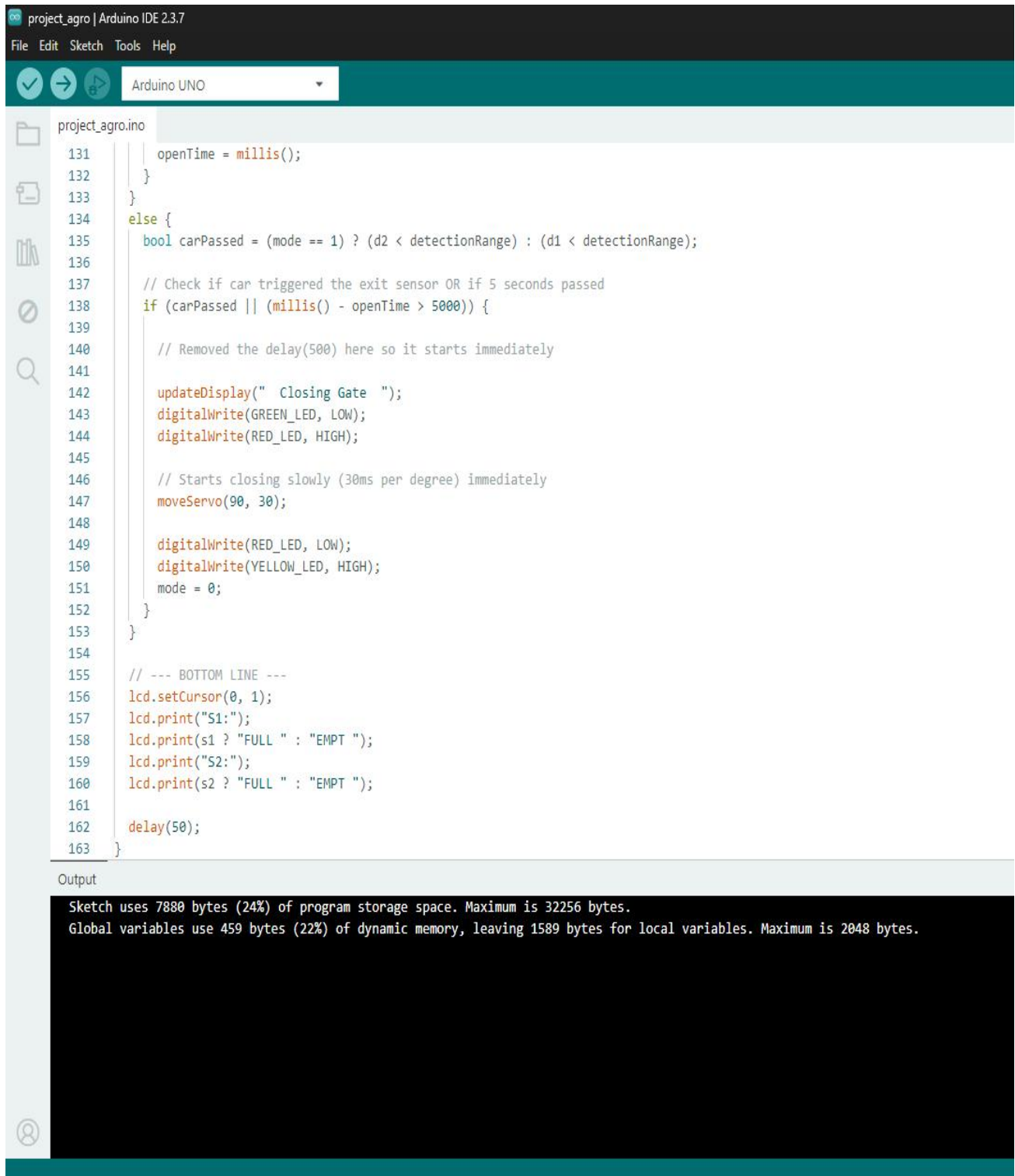


```
project_agro | Arduino IDE 2.3.7
File Edit Sketch Tools Help

project_agro.ino
1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <Servo.h>
4
5  LiquidCrystal_I2C lcd(0x27, 16, 2);
6
7  // Pin Definitions
8  #define TRIG1 2
9  #define ECHO1 3
10 #define TRIG2 4
11 #define ECHO2 5
12 #define SERVO_PIN 11
13 #define YELLOW_LED 6
14 #define RED_LED 7
15 #define GREEN_LED 8
16 #define IR_SLOT1 9
17 #define IR_SLOT2 10
18
19 Servo gateServo;
20
21 int currentPos = 90;
22 int gateState = 0;
23 int mode = 0;
24 bool isFull = false;
25
26 // --- SENSING DISTANCE FIX ---
27 const int detectionRange = 8; // Half of previous distance (was 15, now 7cm)
28
29 unsigned long openTime = 0;
30 unsigned long messageTime = 0;
31 bool messagePending = false;
32
33 void moveServo(int targetAngle, int speedDelay) {
34   if (currentPos < targetAngle) {
35     for (int i = currentPos; i <= targetAngle; i++) {
36       gateServo.write(i);
37       delay(speedDelay);
38     }
39   } else {
40     for (int i = currentPos; i >= targetAngle; i--) {
41       gateServo.write(i);
42       delay(speedDelay);
43     }
44   }
45 }
```

Figure 13: The code window on Arduino IDE

While doing the setup, the project was evaluated that every functionality is accomplished or not. The code was edited as proposed requirements. Next we run the code -



```
project_agro | Arduino IDE 2.3.7
File Edit Sketch Tools Help
[Icons] Arduino UNO

project_agro.ino
131   openTime = millis();
132   }
133   }
134   else {
135     bool carPassed = (mode == 1) ? (d2 < detectionRange) : (d1 < detectionRange);
136
137     // Check if car triggered the exit sensor OR if 5 seconds passed
138     if (carPassed || (millis() - openTime > 5000)) {
139
140       // Removed the delay(500) here so it starts immediately
141
142       updateDisplay(" Closing Gate ");
143       digitalWrite(GREEN_LED, LOW);
144       digitalWrite(RED_LED, HIGH);
145
146       // Starts closing slowly (30ms per degree) immediately
147       moveServo(90, 30);
148
149       digitalWrite(RED_LED, LOW);
150       digitalWrite(YELLOW_LED, HIGH);
151       mode = 0;
152     }
153   }
154
155   // --- BOTTOM LINE ---
156   lcd.setCursor(0, 1);
157   lcd.print("S1:");
158   lcd.print(s1 ? "FULL " : "EMPTY ");
159   lcd.print("S2:");
160   lcd.print(s2 ? "FULL " : "EMPTY ");
161
162   delay(50);
163 }
```

Output

```
Sketch uses 7880 bytes (24%) of program storage space. Maximum is 32256 bytes.
Global variables use 459 bytes (22%) of dynamic memory, leaving 1589 bytes for local variables. Maximum is 2048 bytes.
```

Figure 14: Uploaded code in Arduino IDE

3.9 Cost analysis:

The costs of each individual components are listed here.

Item	Quantity in pieces	Per Price (Taka)	Total Price (Taka)
Arduino UNO Board	1	550	550
16x2 LCD	1	190	190
LED Indicator	4	3	10
Servo Motor (PWM)	1	120	120
I2C LCD Adapter Module	1	30	30
IR Sensor	2	40	80
Ultrasonic Sensor	2	80	160
Connection Wire	35	2	70
Double side tape	1	60	60
Total			1270/=

Here, the total cost is BDT 1270/=. Here we tried to keep the cost minimal and used better components to implement the circuit. This budget will not so over for such a sustainable system with good components. We used less wire and other unnecessary components to complete the project.

3.10 Problem Statement:

1. Parking Lot Problems

Difficulty in Finding Vacant Spaces Quickly finding a vacant space in a multilevel parking lot is difficult if not impossible, especially on weekends or public holidays. Finding spaces during weekends or public holidays can take more than 10 minutes for about 66% of visitors. Stadiums or shopping malls are crowded at peak periods, and difficulty in finding vacant slots at these places is a major problem for customers. Insufficient car park spaces \ lead to traffic congestion and driver frustration.

2. Improper Parking

If a car is parked in such a way that it occupies two parking slots rather than one, this is called improper parking. Improper parking can happen when a driver is not careful about another driver's rights. This is tackled by the development of automated smart car parking system.

Chapter-4: Results And Discussion

1. It guarantees snappy and computerized parking and simple recovery of vehicles.
2. Up to 2-3 cars can be effectively and securely parked in the outlined model.
3. The surface space required is identical to the parking spot of two cars as it were.
4. Most reasonable for parking in workplaces, shopping centers and comparable spots.
5. Low support levels are required by the framework.
6. Sensors utilized have high affectability and are anything but difficult to deal with.
7. Minimal effort framework, giving most extreme computerization.
8. It doesn't require observable pathway operation.
9. Cordial reorientation of cars for driving in and out.
10. Security of vehicle.

4.1 Hardware Connections:

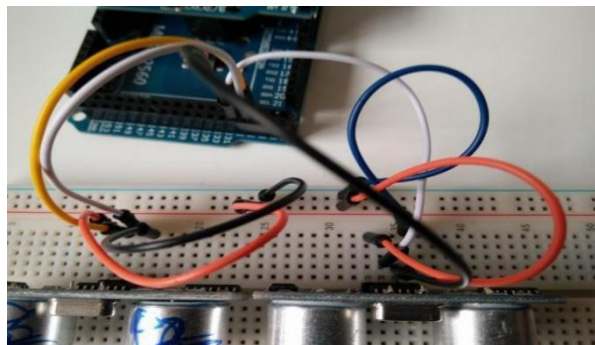


Figure 14: Connection between Arduino and others

- Connect the Vcc stick to the positive rail on your breadboard.
- Connect the Gnd stick to the negative rail on your breadboard.
- Connect the Trig stick to any advanced stick on the Arduino.
- Connect the Echo stick to any computerized stick on the Arduino.
- Finally, interface the positive rail of the breadboard to 5V stick on the Arduino and the negative rail of the breadboard to the Gnd stick on the Arduino.



Figure 16: Actual Working Model

4.2 Simulation analysis:

During test run, all the simulation data were stored. At the initial point of running the circuit, a welcome message is seen in the LCD screen for 1500 milliseconds. After that, the display shows total available free slots at the 1st line. Then all the slot status are updated consecutively as well as the payment status of each slot. There were not so unexpected occurrences but the operation was performing a little bit slower and the servo motor movement was delayed a little though it was not so longer. Sometimes, LCD texts was fluctuating and rendering with delay. But each operation was running finely.

4.3 Measured response/Experimental results:

Though it is a simulation-based project, physical observation was not possible to perform. Thats why here we are not adding any data now. The experiment will be implemented in lab and this section will be updated then.

4.4 Advantage of smart car parking system

1. Reduced traffic
2. Reduced pollution
3. Enhanced User Experience
4. Integrated Payments and POS
5. Increased Safety
6. Real-Time Data and Trend Insight
7. Decreased Management Costs
8. Increased Service and Brand Image
9. Security from theft and car damages
10. Simple structure, simple operation

4.5 Disadvantage of smart car parking system

1. It achieves wireless technology with limited options of connecting to particular device only
2. It does not know who the driver in the car is, checks only the key placed
3. Node-to-node implementation requires more time.

4.6 Benefit analysis of smart car parking system

1. Reduced traffic
2. Reduced pollution
3. Enhanced User Experience
4. Integrated Payments and POS
5. Increased Safety
6. Real-Time Data and Trend Insight
7. Decreased Management Costs
8. Increased Service and Brand Image

Chapter-5: Conclusions And Future Work

5.1 Conclusion

The Smart Car Parking System developed in this project provides a robust, low-cost solution to urban parking challenges. By successfully integrating Arduino UNO with IR and Ultrasonic sensors, the system automates the most time-consuming aspects of parking: searching for a slot and manual gate operation.

Key Achievements:

- **Real-Time Data:** Drivers receive instant updates via an LCD display, eliminating unnecessary driving through full lots.
- **Efficiency:** The automated Servo-controlled gate reduces the need for manual labor and speeds up vehicle entry/exit.
- **Reliability:** The dual-sensor approach ensures that vehicle counts remain accurate, preventing overfilling of the facility.

In summary, this project proves that smart technology can effectively replace traditional, manual parking methods. It offers a sustainable path forward for smart city infrastructure, reducing both traffic congestion and environmental pollution.

5.2 Future Work

To transition this prototype into a full-scale commercial application, the following enhancements are proposed:

- **Mobile App Integration (IoT):** Use Wi-Fi modules (like ESP8266) to allow users to check slot availability and reserve parking spaces from their smartphones.
- **Enhanced Security:** Incorporate *RFID* tags for monthly subscribers and Number Plate Recognition (ANPR) for automated billing and theft prevention.
- **Solar Power:** Implement solar panels to make the system energy-independent and environmentally friendly.
- **Automated Billing:** Integrate digital payment gateways (like bKash or credit card readers) to create a completely contactless transaction experience.
- **Multi-Level Support:** Scale the logic to support multi-story parking garages with floor-by-floor navigation indicators.

REFERENCES

- Thanh Nam Pham¹, Ming-Fong Tsai¹, Duc Bing Nguyen¹, Chyi-Ren Dow¹ and Der-Jiunn Deng². “A Cloud- Based Smart-Parking System Based on Internet of Things Technologies”. IEEE Access, volume 3, pp. 1581 – 1591, September 2015.
- M. Fengsheng Yang, Android Application Development Revelation, China Machine Press, 2010.
- Yanfeng Geng and Christos G. Cassandras. “A New Smart Parking System Based on Optimal Resource Allocation and Reservations”. IEEE Transaction on Intelligent Transportation Systems, volume 14, pp. 1129 -1139, April 2013.
- Cui Shiyao, Wu Ming, Liu Chen, Rong Na. “The Research and Implement of the Intelligent Parking Reservation Management System Based on ZigBee Technology”.
- Website link: <https://www.arduino.cc/en/Main/ArduinoBoardMega>
- Website link: <https://www.engineersgarage.com>

Appendix A:

Real Model :

Figure 16: Parking Process (Car entering parking Spot to park the car).

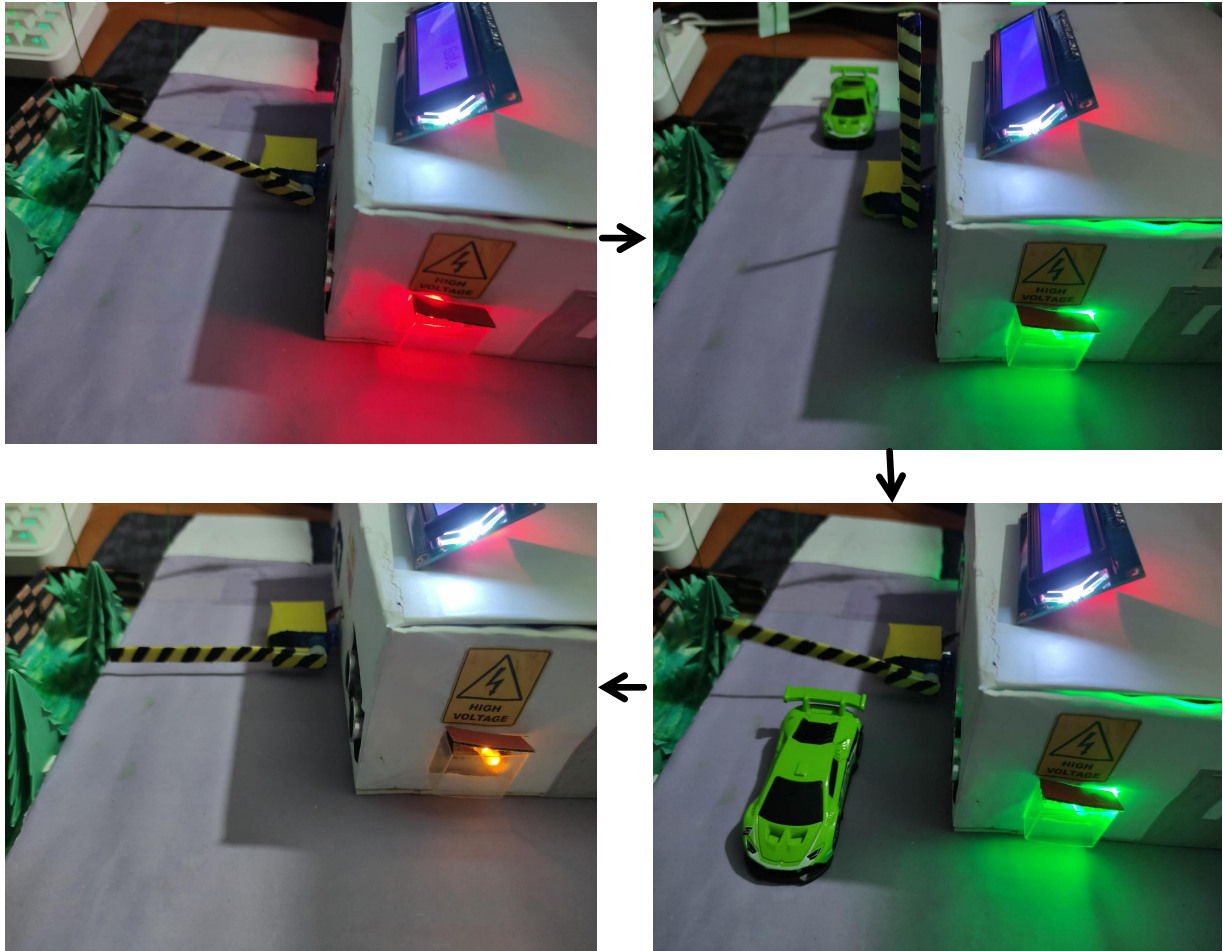


Figure 17: Parking Area



Figure 18: Full Parking Area



Appendix B

Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

// Pin Definitions

#define TRIG1 2
#define ECHO1 3
#define TRIG2 4
#define ECHO2 5
#define SERVO_PIN 11
#define YELLOW_LED 6
#define RED_LED 7
#define GREEN_LED 8
#define IR_SLOT1 9
#define IR_SLOT2 10
```



```

Servo gateServo;

int currentPos = 90;

int mode = 0;    // 0 = Idle, 1 = Entering, 2 = Exiting

const int detectionRange = 8;


unsigned long openTime = 0;
unsigned long messageTime = 0;
bool messagePending = false;


// --- Helper Functions ---
void moveServo(int targetAngle, int speedDelay) {
    if (currentPos < targetAngle) {
        for (int i = currentPos; i <= targetAngle; i++) {
            gateServo.write(i);
            delay(speedDelay);}
    } else {
        for (int i = currentPos; i >= targetAngle; i--) {
            gateServo.write(i);
            delay(speedDelay); } }
    currentPos = targetAngle; }


int getDistance(int trig, int echo) {
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);
    long d = pulseIn(echo, HIGH, 25000);
    int distance = d * 0.034 / 2;
    if (distance <= 0) return 999;
    return distance; }

```

```

void updateDisplay(String msg) {
    lcd.setCursor(0, 0);
    lcd.print("      ");
    lcd.setCursor(0, 0);
    lcd.print(msg);
    messageTime = millis();
    messagePending = true;}

// --- Setup ---
void setup() {
    pinMode(TRIG1, OUTPUT); pinMode(ECHO1, INPUT);
    pinMode(TRIG2, OUTPUT); pinMode(ECHO2, INPUT);
    pinMode(YELLOW_LED, OUTPUT); pinMode(RED_LED, OUTPUT); pinMode(GREEN_LED,
    OUTPUT);

    pinMode(IR_SLOT1, INPUT_PULLUP);
    pinMode(IR_SLOT2, INPUT_PULLUP);

    digitalWrite(YELLOW_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, LOW);

    gateServo.attach(SERVO_PIN);
    gateServo.write(90);
    currentPos = 90;
    lcd.init();
    lcd.backlight();
    updateDisplay(" Parking System");}

// --- Main Loop ---
void loop() {
    bool s1 = (digitalRead(IR_SLOT1) == LOW);

```

```

bool s2 = (digitalRead(IR_SLOT2) == LOW);

bool isFull = (s1 && s2);

int d1 = getDistance(TRIG1, ECHO1);
int d2 = getDistance(TRIG2, ECHO2);

if (messagePending && (millis() - messageTime > 2000)) {
    lcd.setCursor(0, 0);
    lcd.print("  Parking  ");
    messagePending = false;}

// --- GATE LOGIC ---
if (mode == 0) {
    if (d1 < detectionRange) {
        if (isFull) {
            updateDisplay(" PARKING FULL ");
            digitalWrite(YELLOW_LED, LOW);
            digitalWrite(RED_LED, HIGH);
            delay(1500);
            digitalWrite(RED_LED, LOW);
            digitalWrite(YELLOW_LED, HIGH);}
        else {
            updateDisplay(" Entry Detected ");
            digitalWrite(YELLOW_LED, LOW);
            digitalWrite(GREEN_LED, HIGH);
            moveServo(0, 20);
            mode = 1;
            openTime = millis();}}
    else if (d2 < detectionRange) {
        updateDisplay(" Exit Detected ");
        digitalWrite(YELLOW_LED, LOW);
        digitalWrite(GREEN_LED, HIGH);
    }
}

```

```

    moveServo(0, 20);

    mode = 2;

    openTime = millis();
  }}
else {
  bool carPassed = (mode == 1) ? (d2 < detectionRange) : (d1 < detectionRange);

  // Check if car triggered the exit sensor OR if 5 seconds passed
  if (carPassed || (millis() - openTime > 5000)) {

    // Removed the delay(500) here so it starts immediately

    updateDisplay(" Closing Gate ");
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);

    // Starts closing slowly (30ms per degree) immediately
    moveServo(90, 30);

    digitalWrite(RED_LED, LOW);
    digitalWrite(YELLOW_LED, HIGH);
    mode = 0;
  }}
// --- BOTTOM LINE ---
lcd.setCursor(0, 1);
lcd.print("S1:");
lcd.print(s1 ? "FULL " : "EMPT ");
lcd.print("S2:");
lcd.print(s2 ? "FULL " : "EMPT ");
delay(50);
}

```